RUN TIME PERFORMANCE

Mei Nagappan



UNIVERSITY OF WATERLOO | FACULTY OF MATHEMATICS

G4. Provide a Fast Reference Implementation When Asking the Model to Optimize

When asking an AI coding assistant to rewrite a slow function for performance, include a known-fast reference implementation or code pattern from a similar well-optimized context. Framing the task as a transformation from slow-to-fast produces measurably better output than open-ended generation.

Why It Matters

- PIE dataset: models prompted with slow-fast pairs achieve mean speedup of 6.86x and optimize 87.63% of programs
- SBLLM: injecting similar high-performance snippets via BM25 retrieval yields up to 209.59% efficiency improvement over unguided generation
- AUTOPATCH: combining retrieved slow-fast pairs with control-flow-graph context achieves 7.3% execution-efficiency improvement over a strong baseline
- A concrete fast example resolves ambiguity: the model sees the transformation and applies it, rather than hedging toward a readable default
- Without a reference, models choose the laziest transformation (e.g., top-down memoization) rather than the cleanest algorithmic change

Key Practices: Prompting

- Provide the slow function and a fast reference that uses the same algorithmic strategy you want applied (DP table, hash map, pre-sorted structure)
- Explicitly label the two functions: 'Below is a SLOW version followed by a FAST reference. Apply the same transformation to the SLOW version.'
- Ask the model to narrate the mapping: 'Explain which part of the FAST reference corresponds to which part of the new implementation.'
- If you do not have a personal fast reference, describe the optimization pattern in concrete terms (e.g., 'avoid recomputing subproblems by storing results in a 2-D array indexed by (i, j)')

Key Practices: Reviewing

- Check that the model actually applied the transformation from the reference and not reproducing either the slow version or the reference verbatim
- Verify correctness on edge cases: empty input, n=1, inputs with repeated elements
- Measure the speedup on a representative input size. For an algorithmic change (recursive to DP), speedup should grow with n, not be a constant factor
- If the model merged or confused the two versions, ask: 'Lines X-Y still recompute overlapping subproblems. Please fix that.'

G5. Specify Memory Hierarchy Constraints for Low-Level Optimizations

When asking an AI coding assistant to optimize a C/C++ compute loop, always state the relevant hardware constraints: cache size, data type, and access pattern in your prompt. Without this context, the model will produce correct but cache-unfriendly code. Requesting explicit cache reasoning produces structurally different and measurably faster output.

Why It Matters

- LLM-generated code is optimized for readability as it uses simple, idiomatic loop orders that are often cache-hostile
- EffiBench: worst-case memory ratios of up to 43.92x between LLM-generated and human-expert code as memory behavior is not a secondary concern
- LLM-as-a-Critique: models know efficient, allocation-aware patterns but do not select them without explicit structural prompting
- LLM-Vectorizer: 24 out of 149 SIMD transformations generated by models are semantically incorrect as they compile but produce wrong results
- SimdBench: models frequently hallucinate intrinsic function names or produce scalar implementations inside functions claiming to be vectorized

Key Practices: Prompting

- State the cache size and line width if known: 'This runs on x86-64 with 32 KiB L1 cache and 64-byte cache lines'
- Describe the data type and footprint: 'The array holds float values (4 bytes each). An 8x8 tile fits in L1.'
- Name the access pattern you want: 'Restructure the loop so that writes are sequential in memory (row-major for a row-major array)'
- For SIMD or vectorization: ask for only standard C intrinsics or OpenMP SIMD pragmas, not vendor-specific names, and ask it to justify each intrinsic
- Ask for explicit cache reasoning: 'Explain the cache-miss pattern of the original and of your rewrite. Identify the access stride in each.'

Key Practices: Reviewing

- For cache-tiling rewrites, verify correctness by comparing a small tiled result against the untiled result element-by-element (e.g., 64x64 matrix)
- Measure with 'perf stat -e cache-misses,cache-references' (Linux) or Instruments (macOS) to confirm the miss rate dropped
- For SIMD intrinsics, compile with -O2 -mavx2 and check that the binary actually uses vector instructions (objdump -d or godbolt.org)
- Be especially careful if the model changed the loop order on a non-square matrix or on data with aliasing -- verify correctness before benchmarking
- If the model adds prefetch hints (__builtin_prefetch), verify the prefetch distance is reasonable (typically 8-16 iterations ahead for a 64-byte line)

G6. Audit Parallelism Proposals for Correctness Before Measuring Speedup

When an AI coding assistant adds parallelism like OpenMP pragmas, threads, thread pools, or async primitives, treat the output as a correctness-critical draft. Before running performance tests, inspect the code for race conditions, missing synchronization, and false parallelism. Only after the parallel version is verified correct should you measure whether it actually runs faster.

Why It Matters

- ParEval: parallel code generation pass@1 is only 37.8 when compared to 84.1 for serial HumanEval tasks. Parallelism is substantially harder
- Most common failure: missing reduction and private clauses in OpenMP loops, causing data races that produce wrong results silently without crashing
- CONCUR: race conditions and deadlocks are the most frequent bug categories; 'fake concurrency' (no workers ever spawned) is a recurring pattern
- Failure modes: RACET (mutable shared state without locks), HARDT (hardcoded thread counts that do not scale), SPAWN_SEP (thread creation separated from work), models omit 'nowait', generate incorrect 'schedule' clauses, and miss loop-carried dependencies that make parallelization unsafe

Key Practices: Prompting

- Name the shared mutable state explicitly: 'The variable sum is written by multiple iterations. Identify the synchronization strategy needed.'
- Specify the parallel primitive and its correctness guarantees: 'Use #pragma omp parallel for with a reduction clause. Do not use a shared variable in multiple threads write without synchronization.'
- For Python, clarify GIL constraints: 'Use ProcessPoolExecutor for CPU-bound work; ThreadPoolExecutor only for I/O-bound work where threads release the GIL.'
- Ask the model to list every variable read/written in the parallel region and classify each as private, shared-read-only, or shared-written
- Ask for a correctness argument: 'Explain why the output of the parallel version is identical to the serial version on all inputs.'

Key Practices: Reviewing

- For OpenMP: check that every variable written inside a parallel for has a reduction, private, or firstprivate clause, or is protected by a critical or atomic section
- For C++ threads: look for any write to a non-atomic shared variable without a mutex -- this is a data race
- Run the parallel version with a small, deterministic input and compare output to the serial version at least 10 times to catch non-deterministic race conditions
- Check the thread/process count: hardcoded values like std::thread t[4] will hurt on machines with fewer or more cores; prefer omp_get_max_threads() or hardware_concurrency()
- If a Python CPU-bound loop using ThreadPoolExecutor shows no improvement, the GIL is preventing parallel execution -- switch to ProcessPoolExecutor

References

[1] Zhu et al. 'More Than Just Functional: LLM-as-a-Critique for Efficient Code Generation.' NeurIPS 2025. openreview.net/forum?id=0Zri6HSYaK

[2] Peng et al. 'PerfCodeGen: Improving Performance of LLM Generated Code with Execution Feedback.' FORGE@ICSE 2025. arXiv:2412.03578

[3] Huang et al. 'EffiLearner: Enhancing Efficiency of Generated Code via Self-Optimization.' NeurIPS 2024. arXiv:2405.15189

[4] Huang et al. 'EffiBench: Benchmarking the Efficiency of Automatically Generated Code.' NeurIPS 2024. arXiv:2402.02037

[5] Qiu et al. 'How Efficient is LLM-Generated Code? A Rigorous Benchmark (ENAMEL).' NeurIPS 2024. arXiv:2406.06647

[6] Shypula et al. 'Learning Performance-Improving Code Edits.' ICLR 2024. arXiv:2302.07867

[7] Abbassi et al. 'Unveiling Inefficiencies in LLM-Generated Code: Toward a Comprehensive Taxonomy.' ICSME 2025. arXiv:2503.06327

[8] Liu et al. 'What is Wrong with Your Code Generated by Large Language Models?' arXiv:2407.06153, 2024

[9] Madaan et al. 'Self-Refine: Iterative Refinement with Self-Feedback.' NeurIPS 2023. arXiv:2303.17651

References (continued)

[10] Acharya et al. 'Optimizing Code Runtime Performance through Context-Aware RAG.' arXiv:2501.16692, 2025

[11] Gao et al. 'Search-Based LLMs for Code Optimization (SBLLM).' ICSE 2025. arXiv:2408.12159

[12] Taneja et al. 'LLM-Vectorizer: LLM-based Verified Loop Vectorizer.' CGO 2025. arXiv:2406.04693

[13] He et al. 'SimdBench: Benchmarking LLMs for SIMD-Intrinsic Code Generation.' arXiv:2507.15224, 2025

[14] Nichols et al. 'Can Large Language Models Write Parallel Code? (ParEval).' HPDC 2024. arXiv:2401.12554

[15] Huang et al. 'CONCUR: Benchmarking LLMs for Concurrent Code Generation.' arXiv:2603.03683, 2026

[16] Erhabor et al. 'Measuring the Runtime Performance of C++ Code Written by Humans using GitHub Copilot.' arXiv:2305.06439, 2023

[17] Godoy et al. 'An Assessment of LLMs for OpenMP-based Code Parallelization.' Journal of Big Data, 2024. doi:10.1186/s40537-024-01019-z